

EcoRouter — Teleprompter Script v2

Ashwin · Stanford CS 153 · Automation / Agent Systems · June 2026 · 30 regions · A/B · API · Forecast

[0:00 – 0:25] OPENING

Hi, I'm Ashwin.

This is **EcoRouter Agent** — an autonomous system that routes AI compute workloads to the greenest cloud regions in real time, while respecting urgency deadlines, SLA windows, and data residency rules.

I built it for Stanford CS 153, Automation and Agent Systems track, as my One-Person Frontier Lab project.

[0:25 – 1:10] PROBLEM & INSIGHT (3 PTS)

Why does this matter?

As AI scales, the bottleneck is not only GPUs — it's **electricity**. Training and batch inference consume massive power, and grid carbon intensity — grams of CO₂ per kilowatt-hour — **changes by region and by hour** as solar, wind, and demand shift.

Today, a DevOps or FinOps team manually watches grid data and decides where jobs run. That does not scale.

EcoRouter's insight: replace that manual scheduler with an **intelligent agent** — a digital twin infrastructure manager that reads

telemetry, evaluates constraints, and dispatches each job to the best region it can legally reach.

[1:10 – 2:20] EXECUTION & TECHNICAL WORK (5 PTS)

Here is the architecture — four layers, all working end to end.

Layer one — Grid simulator: thirty real AWS regions with live simulated carbon and dollar-per-kilowatt-hour tariffs. Optional Electricity Maps integration for live data.

Layer two — Job queue: realistic enterprise workloads — fine-tuning, RAG batches, GDPR-locked EU inference, HIPAA US jobs. Each job has compute hours, urgent versus flexible priority, and optional locality locks.

Layer three — Agent brain in `ecorouter dot py`: Gemini two-point-five Flash with **LLM tool-calling** — get grid carbon intensity, and assign workload. Plus production routers: **Pareto**, cost-aware, load-balanced, and **forecast-aware deferral** over a twelve-hour window. A mock heuristic fallback guarantees the demo never breaks.

Layer four — This dashboard: Streamlit UI with scrollable carbon charts, region optimizer, A/B baseline comparison, enterprise BYO jobs, ESG PDF export, and a **FastAPI REST layer** for headless integration.

Same logic runs from CLI, API, or UI — fully reproducible from the GitHub repo.

[2:20 – 2:45] DEMO — SETUP

Let me show it live.

At the top: personalized workspace stats — thirty regions, today's greenest region, and carbon upside versus a static us-east-1 baseline.

Scroll the **live carbon chart** — low-carbon Nordic and Canadian hydro versus high-carbon APAC coal regions.

The **job queue** shows realistic scenarios: most jobs are carbon-flexible; a few are locked for GDPR or US latency compliance.

Sidebar: I'll run **Pareto routing** with **forecast-aware scheduling** enabled.

[2:45 – 3:30] DEMO — OPTIMIZATION

The agent has finished routing.

Look at the **Dispatch Summary**. **EcoRouter Carbon** and **Carbon Saved** are in **grams of CO₂**, with concrete totals.

Jobs routed across **multiple green regions** — not one datacenter hotspot — because Pareto load-spreading balances carbon and cost.

The **assignment table** shows baseline versus EcoRouter per job: green rows mean region changed with carbon savings; amber means rerouted; gray means unchanged.

→ *CLICK: Run EcoRouter Optimization · Point at gCO₂ stat cards · Scroll assignment table*

[3:30 – 4:00] DEMO — A/B & FORECAST

Switch to **A/B Comparison**.

EcoRouter is measured against a naive baseline — static us-east-1 or round-robin. You see total carbon, cost in dollars, and **percent savings**. This is hard evidence the agent makes a difference — often fifty to ninety percent carbon reduction on flexible batches.

Open **Forecast and Deferral**: twelve-hour per-region forecast and deferral recommendations for flexible jobs.

→ *CLICK: A/B Comparison tab · Forecast tab*

[4:00 – 4:35] EVALUATION & EVIDENCE (3 PTS)

How do we know it works?

First, **constraint satisfaction**: locality-locked jobs never leave their required region. Urgent jobs respect SLA deadlines.

Second, **quantified savings**: the A/B tab reports grams of CO₂ and dollars saved versus baseline, with per-job breakdowns.

Third, **iteration evidence**: I migrated from deprecated google-generativeai to google-genai, added rate-limit fallback to mock routing, and expanded from four regions to thirty with Pareto and forecast overlays — documented in the README execution log.

The **Tools and API** tab shows curl examples — graders can hit slash api slash v1 slash optimize without the UI.

[4:35 – 5:05] USE CASES & FUTURE (2 PTS)

Who uses this?

Cloud providers and AI labs plug it into batch schedulers to cut Scope 2 emissions without breaking SLAs.

FinOps teams trade carbon against cost with Pareto and cost-aware modes.

DePIN and distributed compute networks apply the same logic wherever workloads are geographically mobile.

If I kept building: production Kubernetes operator, reinforcement learning over long-horizon forecasts, and customer-specific carbon budgets per tenant.

[5:05 – 5:35] PROCESS & DISCLOSURE (2 PTS)

Process and integrity: I used **Cursor and Gemini** as accelerators for boilerplate, debugging, and SDK migration — disclosed in the README. All simulation logic and routing design are original to this project.

[5:35 – 6:00] CLOSE

The full project is open source:

github.com/ashwin122705/ecorouter-agent

TAs and classmates can run it locally with `streamlit run app.py`, or use the deployed demo link in the README.

Thank you for watching.

→ Show GitHub URL on screen. Hold 2 seconds. End recording.

Action Cues — Glance Only

Time	On-Screen Action
0:20	Hero stats: regions, greenest, gCO ₂
2:25	Scroll carbon bar chart
2:35	Job queue — Locality column
2:42	Sidebar: Pareto + Forecast ON
2:45	CLICK: Run EcoRouter Optimization
3:00	Dispatch Summary gCO ₂ cards
3:32	A/B Comparison tab
3:45	Forecast tab
4:10	Tools & API tab
5:40	github.com/ashwin122705/ecorouter-agent

Fallback: ECOROUTER_USE MOCK=1 in .env · **TA guide:** docs/TA_Access_Guide.md